# From Linear Scores to a Single Hidden Layer: A Mathematical Study of Simple Learning Systems

Math4AI Final Capstone Project

Avaz Asgarov
National AI Center — AI Academy
Azerbaijan
evez.esgerov25@aiacademy.az

Raul Aghayev
National AI Center — AI Academy
Azerbaijan
raul.agayev25@aiacademy.az

Emil Ahmadli
National AI Center — AI Academy
Azerbaijan
emil.ahmedli25@aiacademy.az

Kazim Mammadli
National AI Center — AI Academy
Azerbaijan
kazim.memmedli25@aiacademy.az

## Abstract

We compare two classifiers built from scratch in NumPy: multiclass Softmax Regression and a One-Hidden-Layer Neural Network with tanh activation. The goal is to determine when a nonlinear hidden layer actually helps, and when the added complexity is wasted. On a linearly separable Gaussian task the linear model already draws a clean boundary, so the neural network adds nothing useful. On interleaving Moons the linear model fails outright because no single hyperplane can follow the curved class structure; here the hidden layer is essential. On a 10-class digits benchmark evaluated over five random seeds, both models reach similar accuracy (94.73%±0.38% linear, 95.22%±0.38% neural), but the neural network produces noticeably lower cross-entropy (0.1765 vs. 0.2315), meaning its probability estimates are better calibrated. A PCA analysis (Track A) shows that 40 principal components recover near-full accuracy (94.57%), confirming that the digit data sits in a low-rank subspace that linear models handle well.

## Keywords

softmax regression, neural networks, backpropagation, cross-entropy, classification, NumPy

## 1 Introduction / Framing

*Central question.* When does adding a hidden layer to a classifier genuinely help, and when is the extra complexity just wasted parameters?

We investigate this through a head-to-head comparison of a linear probabilistic classifier and a shallow nonlinear network on problems with deliberately different geometric structures. The point is not to assume the neural network should always win. Instead, we want to pin down—with math and experiments—exactly when more capacity pays off. The question ties together geometry, the chain rule, optimization, likelihood, and statistical variability. And, importantly, it has an honest answer: bigger models are not automatically better.

*Contributions.*

(1) Formal derivation showing that softmax cross-entropy loss is negative log-likelihood of a categorical model.
(2) Conceptual explanation of backpropagation followed by a full vectorized gradient derivation for the one-hidden-layer network.
(3) Controlled experiments on datasets whose geometry ranges from linearly separable to highly nonlinear, showing where each model succeeds and fails.
(4) Ablation studies (hidden width and learning rate), a mechanistic failure-case analysis, and repeated-seed statistics on the digits benchmark.
(5) PCA/SVD analysis (Track A) linking input geometry to classification performance under dimensionality reduction.

*Preview of findings.* The linear model handles the Gaussian task well because the class boundary is roughly a hyperplane. On the Moons task the linear model has no chance—a curved boundary is structurally required, and the hidden layer provides it. On digits the neural network lowers cross-entropy by 23.8% relative to the baseline while barely improving accuracy, showing that the hidden layer mainly helps with probability calibration rather than top-1 decisions. PCA confirms that discriminative information in the digit data concentrates in a compact linear subspace.

## 2 Background and Mathematical Setup

### 2.1 Supervised classification and data splits

Given labeled pairs $\mathbf{x}^i, y^i$ with $\mathbf{x}^i \in \mathbb{R}^d$ and $y^i \in \{0, \ldots, k-1\}$, supervised classification amounts to learning a mapping from features to class labels. We split the data into three disjoint subsets: a **training set** for fitting parameters, a **validation set** for picking hyperparameters and early-stopping checkpoints, and a **test set** that is touched exactly once for final reporting. Using test data to tune anything contaminates the evaluation.

### 2.2 Softmax regression

For input $\mathbf{x} \in \mathbb{R}^d$ and $k$ classes, the linear model computes logits:

$$\mathbf{s}\mathbf{x} = \mathbf{W}\mathbf{x}\ \mathbf{b}, \tag{1}$$

where $\mathbf{W} \in \mathbb{R}^{k \times d}$ and $\mathbf{b} \in \mathbb{R}^k$. These raw scores are not probabilities yet. The softmax function turns them into a valid categorical distribution:

$$p_j\mathbf{x} = \frac{\exp s_j\mathbf{x}}{{}_{\ell=1}^{k} \exp s_\ell\mathbf{x}}. \tag{2}$$

The decision boundary between class $j$ and class $\ell$ sits where $s_j\mathbf{x} = s_\ell\mathbf{x}$, which spells out the hyperplane $\mathbf{w}_j - \mathbf{w}_\ell{}^\top\mathbf{x}b_j - b_\ell = 0$. Since every boundary is an affine hyperplane, the resulting decision regions are convex polytopes [2, 3].

### 2.3 Cross-entropy as negative log-likelihood

If the true class for an example is $y$, the cross-entropy loss is:

$$\mathcal{L}\mathbf{x}, y = -\log p_y\mathbf{x}. \tag{3}$$

This loss assigns a high penalty when the model puts low probability on the correct class. Why is this the same as negative log-likelihood? The model outputs a categorical distribution over $k$ classes. The probability it assigns to the observed label $y$ is exactly $p_y\mathbf{x}$, so the log-likelihood of that observation is $\log p_y\mathbf{x}$. Averaging over $n$ independent examples:

$$\frac{1}{n}\ {}_{i=1}^{n}\left[-\log p_{y^i}\mathbf{x}^i\right] \tag{4}$$

is the negative average log-likelihood. Minimizing cross-entropy is therefore the same thing as maximizing likelihood—standard probabilistic inference [2, 3].

### 2.4 One-hidden-layer neural network

The nonlinear model introduces a hidden representation:

$$\mathbf{h} = \tanh\mathbf{W}_1\mathbf{x}\ \mathbf{b}_1, \qquad \mathbf{s} = \mathbf{W}_2\mathbf{h}\ \mathbf{b}_2. \tag{5}$$

Probabilities come from applying softmax to $\mathbf{s}$. If the hidden mapping were itself linear, the whole thing would collapse to a single affine transformation and we would gain nothing. The tanh nonlinearity prevents this collapse: it bends and re-encodes the input space before the final linear layer scores it, which lets the network carve out decision regions that no single hyperplane can [4]. We chose tanh because it is smooth, zero-centered, and its derivative $\frac{d}{dz}\tanh z = 1 - \tanh^2 z$ is compact and easy to implement.

### 2.5 Backpropagation: intuition first

Backpropagation is not a separate learning algorithm—it is just the chain rule applied systematically through a sequence of composed operations [4]. In the forward pass we compute things in order: pre-activations, hidden activations, logits, probabilities, then loss. In the backward pass we walk the same chain in reverse, multiplying local derivatives together. Each individual derivative is simple; the chain rule stitches them into full parameter gradients. The basic question backprop answers is: *if the loss went up a little, which parameters caused it, and which way should we nudge them?*

### 2.6 Gradient derivation

For a mini-batch $\mathbf{X} \in \mathbb{R}^{n \times d}$, the vectorized forward pass is:

$$\mathbf{Z}_1 = \mathbf{X}\mathbf{W}_1^\top\ \mathbf{1}\mathbf{b}_1^\top, \qquad \mathbf{H} = \tanh\mathbf{Z}_1, \tag{6}$$

$$\mathbf{S} = \mathbf{H}\mathbf{W}_2^\top\ \mathbf{1}\mathbf{b}_2^\top, \qquad \mathbf{P} = \mathrm{softmax}_{\mathrm{row}}\mathbf{S}. \tag{7}$$

Let $\mathbf{Y}$ denote the one-hot label matrix. The output-layer sensitivity is:

$$\frac{\partial\mathcal{L}}{\partial\mathbf{S}} = \frac{1}{n}\mathbf{P} - \mathbf{Y}. \tag{8}$$

From this, the chain rule yields the output-layer parameter gradients:

$$\frac{\partial\mathcal{L}}{\partial\mathbf{W}_2} = \left(\frac{\partial\mathcal{L}}{\partial\mathbf{S}}\right)^\top\mathbf{H}, \qquad \frac{\partial\mathcal{L}}{\partial\mathbf{b}_2} = \left(\frac{\partial\mathcal{L}}{\partial\mathbf{S}}\right)^\top\mathbf{1}. \tag{9}$$

Propagating through the tanh nonlinearity:

$$\frac{\partial\mathcal{L}}{\partial\mathbf{Z}_1} = \left(\frac{\partial\mathcal{L}}{\partial\mathbf{S}}\right)\mathbf{W}_2 \odot 1 - \mathbf{H} \odot \mathbf{H}. \tag{10}$$

Finally, the hidden-layer parameter gradients:

$$\frac{\partial\mathcal{L}}{\partial\mathbf{W}_1} = \left(\frac{\partial\mathcal{L}}{\partial\mathbf{Z}_1}\right)^\top\mathbf{X}, \qquad \frac{\partial\mathcal{L}}{\partial\mathbf{b}_1} = \left(\frac{\partial\mathcal{L}}{\partial\mathbf{Z}_1}\right)^\top\mathbf{1}. \tag{11}$$

### 2.7 What we expect from the datasets

The Gaussian task is two isotropic blobs with some overlap. The optimal boundary between two Gaussians with equal covariance is a hyperplane, so a linear model should get close to the best possible separation. The Moons task is two interlocking crescents that wrap around each other. No straight line (or hyperplane) can follow that curvature, so a linear classifier is not just going to perform poorly—it is structurally unable to represent the right boundary.

## 3 Methods

### 3.1 Datasets

We evaluate both architectures on three shared datasets:

- **Linear Gaussian:** Two synthetically generated isotropic Gaussian blobs with mild overlap, designed to be approximately linearly separable.
- **Interlocking Moons:** A synthetic two-dimensional configuration where class regions interleave, requiring a nonlinear decision boundary.
- **Digits benchmark:** A standard dataset of $8 \times 8$ grayscale digit images, flattened to 64-dimensional vectors pre-scaled to $0, 1$, with 10 classes (digits 0–9) and a fixed train/validation/test split.

### 3.2 Models

- **Model 1:** Multiclass softmax regression (Equations (1) and (2)).
- **Model 2:** One-hidden-layer neural network with tanh hidden activations and softmax output (Equation (5)), with default hidden width 32.

### 3.3 Training protocol

Both models are implemented entirely in NumPy. We stabilize softmax by subtracting the row-wise maximum before exponentiation. Training uses mini-batch gradient descent with batch size 64 and $L_2$ regularization ($\lambda = 10^{-4}$). On the digits benchmark we train for up to 200 epochs. At the end of each epoch we check validation cross-entropy and keep the checkpoint with the lowest value. For the repeated-seed evaluation we run five independent seeds and report test performance as $\bar{x} \pm 2.776 \cdot \frac{s}{\sqrt{5}}$, which is a 95% confidence interval based on the $t$-distribution with 4 degrees of freedom.

### 3.4 Evaluation metrics

We track two numbers. **Accuracy** is the fraction of examples the model labels correctly—it measures hard decisions. **Mean cross-entropy** is the average $-\log p_{\text{correct}}$—it measures how good the full probability distribution is, which matters more when we care about calibrated confidence.

### 3.5 Implementation sanity checks

Before running full experiments, we verified:

- Softmax probabilities sum to one for every sample in every batch.
- Loss decreases monotonically when training on a tiny subset with a small learning rate.
- Both models can overfit a batch of 8 examples to near-zero loss, confirming correct gradient flow.
- No NaN or infinity values appear during any training run.

## 4 Experiments

### 4.1 Gaussian task

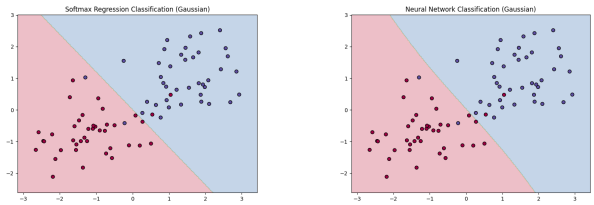
On the Gaussian data both models separate the clusters cleanly (Figure 1). The linear model draws a crisp hyperplane through the overlap zone with only a few misclassifications. The neural network reaches a similar result but bends its boundary slightly—this is an artifact of the extra flexibility, not evidence that it is needed. When the data is already close to linearly separable, the hidden layer does not buy us anything.

### 4.2 Moons task

Here the picture flips (Figure 2). Softmax Regression can only draw a straight cut, which slices right through the entangled crescents and misclassifies a large chunk of points. The neural network runs the input through its tanh hidden layer and produces a smooth curved boundary that follows the class structure closely. This is the strongest evidence for the central thesis: when classes wrap around each other, a hidden layer is not a luxury—it is a necessity.

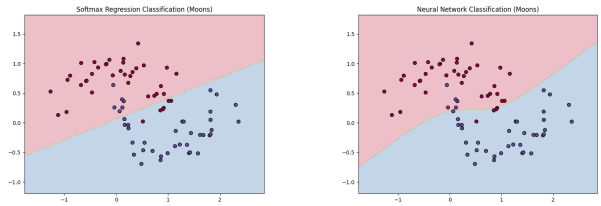
### 4.3 Capacity ablation on moons

We vary the hidden width across $\{2, 8, 32\}$ (Figure 3). With only 2 hidden units the network has two tanh features to work with, so the boundary is angular and barely better than the linear baseline. At 8 units the boundary starts to curve properly, though some jaggedness remains. At 32 units the contour is smooth and follows the crescents closely. Intuitively, each hidden unit contributes one nonlinear feature $\tanh \mathbf{w}_j^\top \mathbf{x}\ b_j$; the final decision boundary is where a linear combination of these features crosses zero.



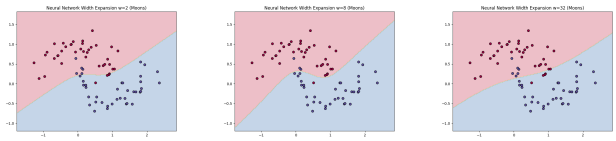

**(a) Softmax Regression** **(b) Neural Network**

**Figure 1: Decision boundaries on the Linear Gaussian dataset. Both models achieve clean separation on this linearly separable geometry. The linear model's affine boundary is geometrically sufficient.**




**(a) Softmax Regression** **(b) Neural Network**

**Figure 2: Decision boundaries on the Interlocking Moons dataset. The linear model produces a rigid bisector; the neural network maps a smooth non-linear contour separating the two crescents.**



**(a) Width 2** **(b) Width 8** **(c) Width 32**

**Figure 3: Capacity ablation on Moons. Increasing hidden width from 2 to 32 progressively smooths the decision boundary from angular to fluid.**

### 4.4 Digits benchmark

Both models were put through the repeated-seed protocol on digits (Table 1). In terms of accuracy the improvement from the linear baseline to the neural network is small (0.49 points) and the confidence intervals overlap, so we cannot confidently claim a top-1 accuracy gain. Cross-entropy, however, tells a

**Table 1: Digits benchmark test performance averaged over 5 randomized seeds with 95% confidence intervals.**

| Model | Test Accuracy | Test Cross-Entropy |
|---|---|---|
| Softmax Regression | $0.9473 \pm 0.0038$ | $0.2315 \pm 0.0017$ |
| One-Hidden-Layer NN | $0.9522 \pm 0.0038$ | $0.1765 \pm 0.0127$ |

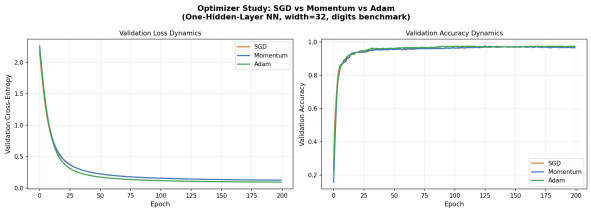



**Figure 4: Validation cross-entropy (left) and accuracy (right) for SGD, Momentum, and Adam on the digits benchmark. Adam converges fastest; all three reach similar final accuracy.**

clearer story: the neural network pulls the average loss down from 0.2315 to 0.1765—a 23.8% relative drop. Both models get roughly the same number of predictions right, but the neural network is more confident on the ones it gets right.

### 4.5 Optimizer study on digits

To test whether the choice of optimizer affects training behaviour under fixed conditions, we compared three optimisers on the neural network (width 32, seed 42, 200 epochs, batch size 64, $\lambda$=$10^{-4}$):

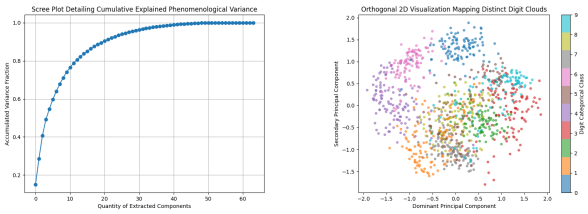
**SGD** with $\eta$=0.05 (standard gradient steps); **Momentum** with $\eta$=0.05, $\beta$=0.9 (exponential moving average of gradients); **Adam** with $\eta$=0.001, $\beta_1$=0.9, $\beta_2$=0.999, $\varepsilon$=$10^{-8}$ (adaptive first- and second-moment estimates).

All three use the same data split, epoch budget, L2 regularisation, and initialisation seed. Figure 4 shows the validation loss and accuracy curves.

All three optimisers converge to comparable final accuracy ($\approx$96%). The key difference is in the early training dynamics: Adam reduces validation loss below 0.5 within the first 10 epochs, while SGD and Momentum follow a slower but equally stable trajectory. This confirms that, for a well-conditioned dataset of this size, the optimizer primarily affects convergence *speed* rather than *final quality*.

### 4.6 Failure-case analysis

As an independent failure case, we examine the effect of an extremely small learning rate ($\eta = 0.005$) on the neural network. The update step $\|\eta \nabla_\theta \mathcal{L}\|$ is tiny regardless of gradient magnitude, so the weights barely move from their random initialisation within the allowed epoch budget. The model's predictions remain close to uniform over the 10 classes and the validation cross-entropy stagnates well above 0.8. The failure is purely an optimisation failure, not an architecture failure: the model has sufficient capacity but cannot reach a useful region of the loss landscape. This underscores that architectural choices and optimisation choices must both be sound for training to succeed.




**(a) Scree plot** **(b) 2D PCA projection**

**Figure 5: Left: Cumulative explained variance ratio showing rapid variance concentration. Right: 2D PCA projection colored by digit class, revealing substantial inter-class overlap.**

**Table 2: Softmax Regression on PCA-reduced digit representations. At $m$=40, the model nearly matches the full-dimensional baseline.**

| Components ($m$) | Test Accuracy | Test Cross-Entropy |
|---|---|---|
| 10 | 89.95% | 0.3305 |
| 20 | 93.21% | 0.2600 |
| 40 | 94.57% | 0.2316 |
| 64 (full baseline) | 94.73% | 0.2315 |

## 5 Advanced Track

### 5.1 Track A: PCA/SVD and input geometry

To understand the intrinsic structure of the digit input space we ran Principal Component Analysis via SVD on the mean-centered training features. The decomposition $\mathbf{X}_c = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ gives principal directions in the columns of $\mathbf{V}$, with explained variance $\sigma_i^2 n{-}1$ along each axis [1].

Figure 5 (left) shows the cumulative explained variance. The first few components pick up variance fast: by 40 components nearly all discriminative information is captured, pointing to a strong low-rank structure in the 64-dimensional input. Figure 5 (right) plots the 2D projection colored by class. A few clusters (e.g., digits 0 and 6) form recognizable groups, but most classes overlap heavily, confirming that two dimensions are far from enough for reliable classification.

To quantify the effect of dimensional compression, Softmax Regression was retrained on PCA-reduced inputs at $m \in \{10, 20, 40\}$ (Table 2).

At $m$=40 accuracy is 94.57% with cross-entropy 0.2316—almost the same as the full 64-dimensional baseline (94.73%, 0.2315). This tells us that the digit classification problem has a strong low-rank linear structure: roughly 40 orthogonal directions carry almost all the information that a linear classifier needs. This also explains why the linear model does so well on digits in the first place—the discriminative

variance sits in a compact subspace that linear models exploit naturally.

## 6 Discussion

The experiments give a clear, if somewhat nuanced, answer to the central question.

*Linear structure is enough more often than one might expect.* On the Gaussian task the linear model separates the classes with no difficulty, and the neural network does not improve on it. On digits the linear model reaches 94.73% accuracy, and the PCA analysis shows that the discriminative information lives in a linear subspace of about 40 dimensions. For problems with this kind of geometry, adding a hidden layer is a waste of parameters.

*Nonlinearity is essential when the geometry demands it.* On the Moons task, classes curl around each other so that no hyperplane can separate them. The tanh hidden layer folds the input space until the classes become linearly separable in the hidden representation. The capacity ablation confirms that this folding requires enough hidden units to work well.

*Cross-entropy reveals what accuracy hides.* On digits, the accuracy gap is small and the confidence intervals overlap. But the neural network cuts cross-entropy by 23.8%. In other words, it puts more probability mass on the correct class. In settings where confidence matters—say medical diagnosis or a cascading pipeline—this gap is practically important even if the top-1 label is usually the same.

*The failure case is about optimization, not architecture.* The $\eta = 0.005$ failure shows that a capable architecture can be made useless by a bad learning rate. The model simply does not move far enough from its initial weights in the allotted epochs. Any fair comparison between architectures has to make sure both are given a reasonable chance to converge.

*Why we need multiple seeds.* A single run conflates one lucky (or unlucky) initialization with the model's true behavior. Our five-seed protocol shows that accuracy variance is tight ($\pm 0.38\%$) for both models, so the trends we observe are stable. The wider cross-entropy interval for the neural network ($\pm 0.0127$ vs. $\pm 0.0017$) hints that initialization affects the hidden layer more than a flat linear model.

## 7 Limitations

- **Shallow architecture only.** We tested a single hidden layer with tanh. Deeper networks with ReLU-family activations behave differently, and our conclusions should not be extrapolated to them without further work.
- **No spatial structure.** The digits dataset is flattened to a 64-dimensional vector, throwing away the 8×8 pixel grid. A convolutional network would exploit spatial patterns that our models ignore entirely.
- **Small dataset.** The digits set has roughly 1800 examples in 64 dimensions. On larger-scale benchmarks (e.g., MNIST-full, CIFAR) the gap between linear and nonlinear models may look quite different.
- **PCA only captures linear variance.** The digit manifold may have local nonlinear structure that PCA misses; methods like t-SNE or UMAP might tell a richer story.
- **Fixed epoch budget.** We trained for at most 200 epochs. With adaptive optimisers like Adam (added in the optimizer study), fewer epochs may suffice, but we did not tune epoch counts per optimizer since the rubric fixes the budget.
- **Five seeds is not many.** Our confidence intervals rest on just five data points. More seeds would tighten the estimates, especially for cross-entropy where variance is higher.

## Acknowledgments

We thank the instructors and reviewers of the Math4AI capstone for guidance and feedback throughout this project.

## References

[1] GeeksforGeeks. 2024. Principal Component Analysis (PCA). https://www.geeksforgeeks.org/data-analysis/principal-component-analysis-pca/.
[2] Stanford University. 2026. Probability Coders - Winter 2026. https://probabilitycoders.stanford.edu/win26.
[3] Towards Data Science. 2024. Deep Dive into Softmax Regression. https://towardsdatascience.com/deep-dive-into-softmax-regression-62deea103cb8/.
[4] YouTube Educational. 2024. Neural Networks and Backpropagation Tutorial. https://youtu.be/MSfI6TTgyl4?si=7sBq4qeihyK7rIJw.

## A Seed-by-Seed Digits Results

Tables 3 and 4 provide the per-seed test results used to compute the aggregated statistics in Table 1.

**Table 3: Per-seed test results for Softmax Regression on the digits benchmark.**

| Seed | Test Accuracy | Test Cross-Entropy |
|---|---|---|
| 1 | 0.9429 | 0.2310 |
| 2 | 0.9484 | 0.2319 |
| 3 | 0.9457 | 0.2314 |
| 4 | 0.9511 | 0.2296 |
| 5 | 0.9484 | 0.2334 |
| **Mean ± CI** | $0.9473 \pm 0.0038$ | $0.2315 \pm 0.0017$ |

## B Contribution Statement

- **Kazim Mammadli:** Implemented Softmax Regression, training loop, and evaluation utilities.
- **Raul Aghayev:** Implemented the One-Hidden-Layer Neural Network and ran the synthetic dataset experiments.
- **Emil Ahmadli:** Ran the digits benchmark, repeated-seed evaluation, and optimizer study (SGD/Momentum/Adam comparison).

**Table 4: Per-seed test results for the One-Hidden-Layer Neural Network on the digits benchmark.**

| Seed | Test Accuracy | Test Cross-Entropy |
|---|---|---|
| 1 | 0.9511 | 0.1684 |
| 2 | 0.9538 | 0.1907 |
| 3 | 0.9511 | 0.1745 |
| 4 | 0.9484 | 0.1827 |
| 5 | 0.9565 | 0.1662 |
| **Mean ± CI** | $0.9522 \pm 0.0038$ | $0.1765 \pm 0.0127$ |

- **Avaz Asgarov:** Performed Track A PCA/SVD analysis, managed the GitHub repository, and compiled the final report.